

Recognizing the 15 Patterns

HPR interview study guide. The goal isn't to solve — it's to look at a problem and instantly say "that's a ____." Read this like flashcards.

The whole skill in one shift: stop asking "how would I solve this?" (that's inventing brute force). Ask "which of my 15 shapes is this?" (that's recognizing). The reflex:
① **bound on n** → allowed complexity (kills brute force) ② **cue words** → 2–3 candidates ③ **a constraint** kills the impossible ones (e.g. negatives kill sliding window).

The 15 patterns — cue · what it does · the tell

1. Two-pointer. CUE: sorted (or sortable) array, "find a pair/triple", "is there an $x+y = t$ ".

DOES: one index from each end, walk them toward each other.

TELL: sorted + looking for a relationship between two elements. *Ex: Two Sum on a sorted array → lo/hi pointers, sum too small move lo up.*

2. Sliding window. CUE: "longest/shortest **contiguous** subarray/substring with a property", values **non-negative**.

DOES: grow a window from the right, shrink from the left, keep a running quantity.

TELL: contiguous + the running quantity changes *monotonically with window size* (adding never decreases it, removing never increases it). *Ex: longest substring with all-distinct chars; shortest subarray with sum $\geq S$ (positive ints).*

3. Prefix-sum array (range queries). CUE: many range-sum queries on a **fixed** array, "sum of range `[i, j]`", 2D rectangle sums, "immutable". The endpoints are **known**.

DOES: precompute running totals once ($O(n)$); any range sum = `S[j] - S[i]` in $O(1)$.

TELL: repeated range sums, 2D grid rectangle sums, or many range-add updates (difference array — the inverse trick). *You already know the endpoints; you're not searching for them.*

4. Prefix sum + hashmap. CUE: "contiguous subarray summing to **exactly k**", especially when values can be **negative**.

DOES: running prefix sum `S`; count earlier prefixes equal to `S - k` via a hashmap.

TELL: "sum = k" + endpoints *unknown* (and negatives *break* the window, so you fall back to this). *Seed the map with {0:1}.*

5. Hashmap / frequency array. CUE: "count occurrences", "seen before", "distinct", "anagram", "duplicate".

DOES: map value → count (or last-seen index). $O(1)$ lookup.

TELL: you need to remember what you've seen. *Bounded values (e.g. lowercase letters) → use a plain `int[26]`, not a hashmap.*

6. Sorting + sweep. CUE: "intervals/meetings overlap or merge", "schedule", anything about order on a number line.

DOES: sort by start, then walk once keeping a "current" interval.

TELL: the word "interval" almost always = sort first. *Ex: merge overlapping intervals.*

7. Binary search (on a sorted array). CUE: already sorted + "find / insert position of a value".

DOES: halve the search range each step.

TELL: sorted data + you're locating a specific value. $O(\log n)$. Use `mid = lo + (hi - lo) / 2`.

8. Binary search on the answer. CUE: "min/max value such that a property holds", "is it possible within budget K?", and the property is **monotonic** (once true, stays true).

DOES: binary-search the *answer space*, test feasibility at each midpoint.

TELL: huge answer range + a yes/no feasibility check. *Ex: minimum capacity to ship in D days.*

9. Heap (priority queue). CUE: "k-th largest/smallest", "top k", "running median", "merge k lists".

DOES: keep only the k elements that matter, $O(\log k)$ per op.

TELL: you want the best few, not a full sort. *Sorting also works ($O(n \log n)$) but heap ($O(n \log k)$) / quickselect ($O(n)$) is the faster gear — say so.*

10. Monotonic stack. CUE: "next greater/smaller element", "span", "largest rectangle in histogram".

DOES: a stack kept in increasing/decreasing order; pop while the new element breaks the order.

TELL: "next/previous greater or smaller" is the dead giveaway.

11. Stack. CUE: "balanced", "valid parentheses", nesting, undo, "evaluate expression".

DOES: push openers, pop/match on closers; must be empty at the end.

TELL: matching/nesting that must close in reverse order. In C: `char st[n]; int top=0;`

12. BFS (breadth-first search). CUE: "minimum steps/moves to reach", unweighted shortest path, "levels", grid distances.

DOES: explore level by level with a queue; first arrival = fewest steps.

TELL: graph/grid + *shortest*. *Queue*.

13. DFS / union-find. CUE: "connected components", "flood fill", "number of islands", "all paths", cycle detection.

DOES: go deep then backtrack (recursion/stack); union-find for grouping.

TELL: graph/grid + *reach/group everything* (not shortest).

14. Dynamic programming. CUE: "number of ways", "min/max cost/length", "can you make X", overlapping subproblems, small-ish n with combinatorial feel.

DOES: define a state + a transition; fill a table.

TELL: the answer depends on answers to smaller versions of the same problem. *Ex: coin change, edit distance, longest increasing subsequence.*

15. Math / bit trick. CUE: huge n with arithmetic, divisibility/parity, "without extra space", "single number".

DOES: closed-form formula, GCD, XOR tricks, sieve.

TELL: n is astronomically large ($1e9$ – $1e18$) so only $O(\log n)/O(1)$ survives → must be math.

The trap pairs (where you got bitten today)

Two problems that look the same; a single constraint decides. This is the real test.

#3 Prefix-sum array vs #4 Prefix-sum+hashmap

Both use running totals. The decider: **do you know the endpoints?** Endpoints *known* ("sum of range [i,j]", repeated queries, 2D) → plain array, $O(1)$ per lookup. Endpoints *unknown* ("which subarrays sum to k?") → hashmap to find them in $O(n)$ instead of $O(n^2)$. Same $O(n)$ storage; opposite direction (range → sum vs sum → range).

#2 Sliding window vs #4 Prefix-sum+hashmap

Both scream "contiguous subarray." The decider: **are negatives allowed?** Non-negative → window (running quantity changes monotonically with window size). Negatives present → window is *dead* (adding can lower the sum, breaking that monotonicity) → prefix-sum + hashmap.

#1 Two-pointer vs #12/13 BFS/DFS

"Find a pair" on a flat **array** = two-pointer. BFS/DFS only when there's a **graph or grid** (nodes, edges, neighbors). No graph → no BFS.

#7 Binary search vs #8 Binary search on answer

Searching for a value *in sorted data* = #7. Searching for the smallest/largest *parameter that makes something feasible* = #8 (the array may be unsorted; you bisect the answer range, not the array).

#9 Heap vs full sort

"k-th largest" — sorting works and is a fine first answer ($O(n \log n)$). But you only need the top k, so heap ($O(n \log k)$) or quickselect ($O(n)$) is faster. When they ask "can you do better?", that's the answer.

#6 Sort+sweep vs #5 Hashmap

Intervals/overlap = order on a number line → **sort**. A hashmap has no order, so it can't help with overlap. If your complexity instinct says " $n \log n$ ", that's the sort talking — follow it.

Recognition self-quiz (do it in the dark)

Read the cue, name the pattern + number before peeking. Answers at the bottom.

1. "Longest substring with at most 2 distinct characters." ($n \leq 1e5$)
2. "Count subarrays summing to k; array has negatives." ($n \leq 1e5$)
3. "Minimum number of moves for a knight to reach a square." (grid)
4. "Largest rectangle in a histogram."
5. "k-th smallest element in an unsorted array." ($n \leq 1e5$)
6. "Given sorted array, find if two values sum to t."
7. "Number of islands in a grid of land/water."
8. "Smallest ship capacity to deliver all packages within D days."
9. "Are these parentheses/brackets valid?"

10. "How many distinct values appear?" (values up to $1e9$)
11. "Merge all overlapping meeting times."
12. "Coins of given denominations — fewest to make amount N ."
13. "Find the one number that appears once; all others appear twice."
14. "For each element, the next element greater than it."
15. "Shortest contiguous subarray with sum $\geq S$; all positive." ($n \leq 1e5$)
16. "Answer Q queries, each the sum of a fixed array over range $[i, j]$." (immutable)

Answers: 1) #2 sliding window · 2) #4 prefix-sum+hashmap (negatives!) · 3) #12 BFS · 4) #10 monotonic stack · 5) #9 heap/quickselect (sort = OK first answer) · 6) #1 two-pointer · 7) #13 DFS/union-find · 8) #8 binary search on the answer · 9) #11 stack · 10) #5 hashmap (values huge → not an array) · 11) #6 sort + sweep · 12) #14 DP · 13) #15 bit trick (XOR all) · 14) #10 monotonic stack · 15) #2 sliding window (positive → monotonic) · 16) #3 prefix-sum array (known endpoints, repeated queries).

If a cue doesn't instantly fire the pattern, that's the one to drill tomorrow. Recognition over recall — let it wash over you. Sleep when it's boring.